Visualize circuits

Package versions

It's often useful to see the circuits you're creating. Use the following options to display Qiskit circuits.

```
from qiskit import QuantumCircuit
```

Draw a quantum circuit

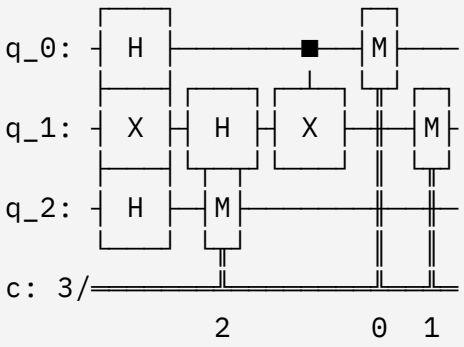
The `QuantumCircuit` class supports drawing circuits through the `draw()` method, or by printing the circuit object. By default, both render an ASCII art version of the circuit diagram.

Note that `print` returns `None` but has the side effect of printing the diagram, whereas `QuantumCircuit.draw` returns the diagram with no side effects. Since Jupyter notebooks display the output of the last line of each cell, they appear to have the same effect.

```
# Build a quantum circuit
circuit = QuantumCircuit(3, 3)
circuit.x(1)
circuit.h(range(3))
circuit.cx(0, 1)
circuit.measure(range(3), range(3));
```

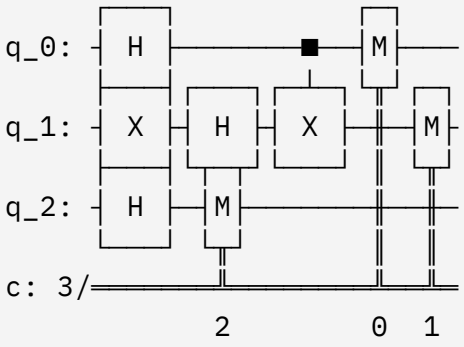
```
print(circuit)
```

Output:




```
circuit.draw()
```

Output:



Alternative renderers

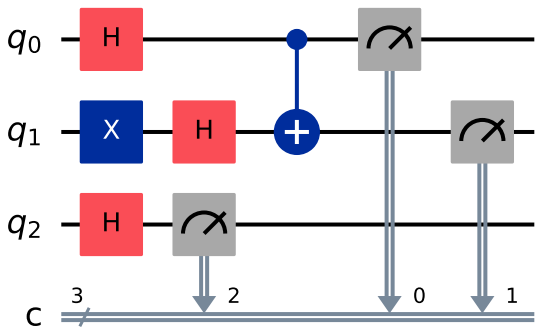
A text output is useful for quickly seeing the output while developing a circuit, but it doesn't provide the most flexibility. There are two alternative output renderers for the quantum circuit. One uses Matplotlib ↗ and the other uses LaTeX ↗. The LaTeX renderer requires the qcircuit package ↗. Select these renderers by setting the "output" argument to the strings `mpl` and `latex`.

Tip

OSX users can get the required LaTeX packages through the mactex package ↗.

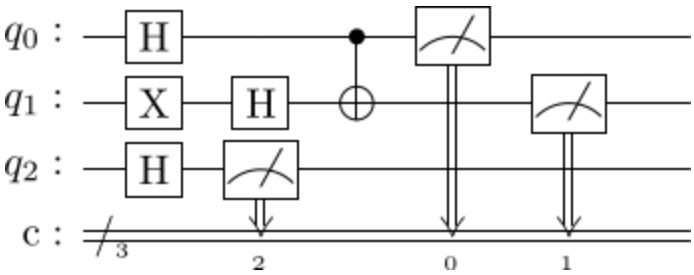
```
# Matplotlib drawing
circuit.draw(output="mpl")
```

Output:




```
# Latex drawing
circuit.draw(output="latex")
```

Output:

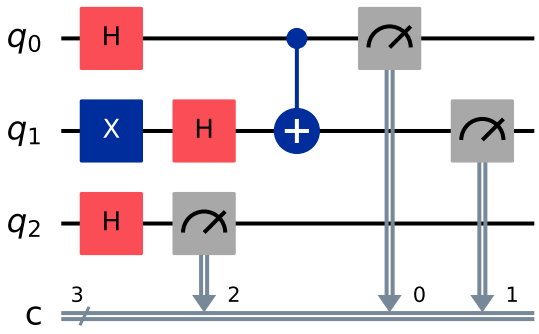


Save output

Drawing a large-scale circuit inline in a Jupyter notebook can be slow or unreadable. You can save the diagram directly to a file, then open it in an image viewer and zoom in as needed.

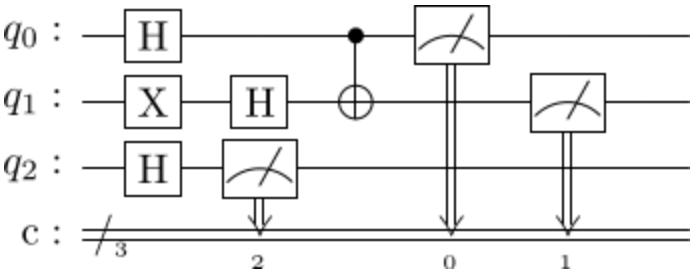
```
# Save as an image using the Matplotlib drawer
circuit.draw(output="mpl", filename="circuit-mpl.jpeg")
```

Output:




```
# Or save a LaTeX rendering
circuit.draw(output="latex", filename="circuit-latex.pdf")
```

Output:



Control circuit drawings

By default, the `draw()` method returns the rendered image as an object and does not output anything. The exact class returned depends on the output specified: `'text'` (the default) returns a `TextDrawer` object, `'mpl'` returns a `matplotlib.Figure` object, and `latex` returns a `PIL.Image` object. Jupyter notebooks understand these return types and render them properly, but when running outside of Jupyter, images will not display automatically.

The `draw()` method has optional arguments to display or save the output. When specified, the `filename` kwarg takes a path to which it saves the rendered output. Alternatively, if you're using the `mpl` or `latex` outputs, you can use the `interactive` kwarg to open the image in a new window (this will not always work from within a notebook).

Customize the output

Depending on the output, there are also options to customize the circuit diagram.

Disable plot barriers and reverse bit order

The first two options are shared among all three backends. They allow you to configure both the bit orders and whether or not you draw barriers. These can be set by the `reverse_bits` kwarg and `plot_barriers` kwarg, respectively. The following examples work with any output renderer; `mpl` is used here for brevity.

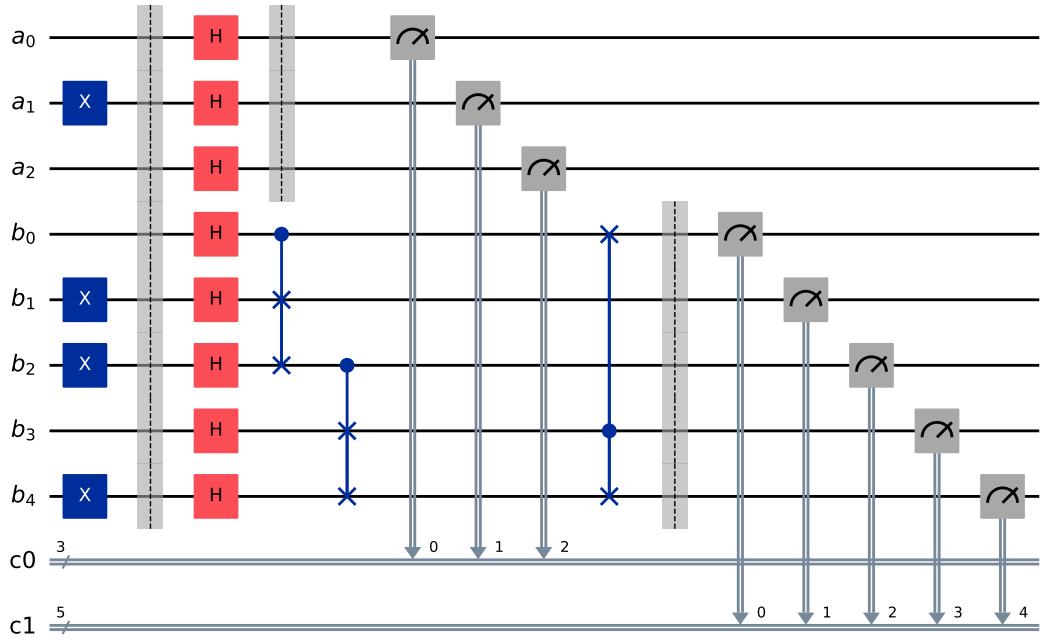
```
from qiskit import QuantumRegister, ClassicalRegiste

# Draw a new circuit with barriers and more registers
q_a = QuantumRegister(3, name="a")
q_b = QuantumRegister(5, name="b")
c_a = ClassicalRegister(3)
c_b = ClassicalRegister(5)

circuit = QuantumCircuit(q_a, q_b, c_a, c_b)
circuit.x(q_a[1])
circuit.x(q_b[1])
circuit.x(q_b[2])
circuit.x(q_b[4])
circuit.barrier()
circuit.h(q_a)
circuit.barrier(q_a)
circuit.h(q_b)
circuit.cswap(q_b[0], q_b[1], q_b[2])
circuit.cswap(q_b[2], q_b[3], q_b[4])
circuit.cswap(q_b[3], q_b[4], q_b[0])
circuit.barrier(q_b)
circuit.measure(q_a, c_a)
circuit.measure(q_b, c_b);
```

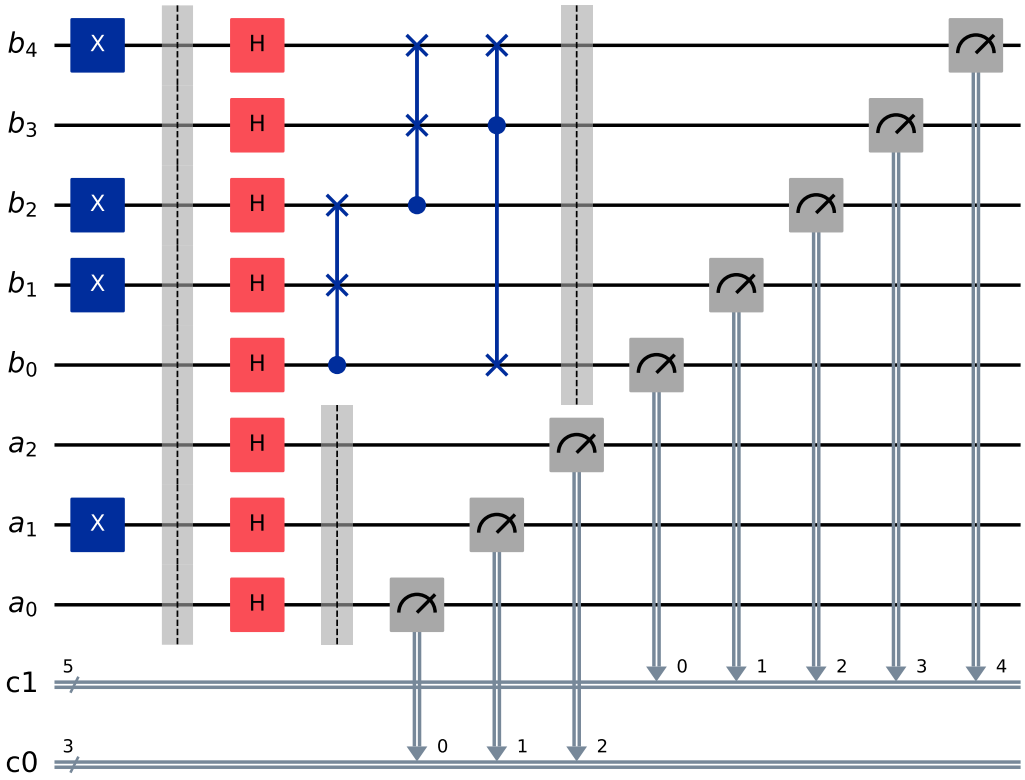
```
# Draw the circuit
circuit.draw(output="mpl")
```

Output:



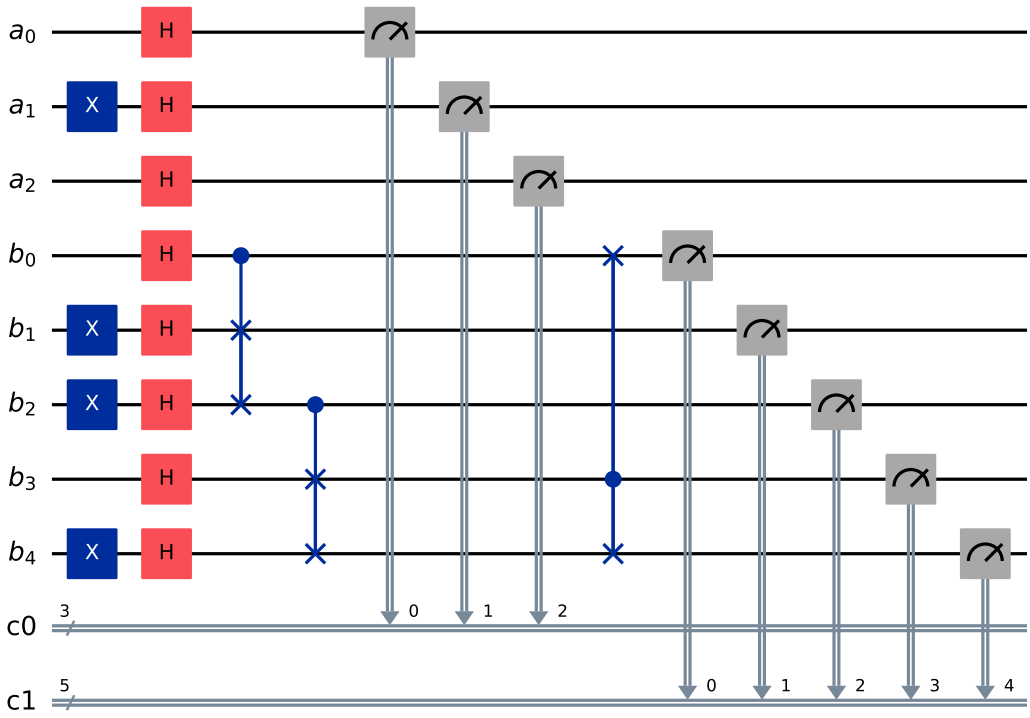

```
# Draw the circuit with reversed bit order
circuit.draw(output="mpl", reverse_bits=True)
```

Output:



```
# Draw the circuit without barriers
circuit.draw(output="mpl", plot_barriers=False)
```

Output:



Renderer-specific customizations

Some available customizing options are specific to a renderer.

The `fold` argument sets a maximum width for the output. In the `text` renderer, this sets the length of the lines of the diagram before it is wrapped to the next line. When using the 'mpl' renderer, this is the number of (visual) layers before folding to the next line.

The `mpl` renderer has the `style` kwarg, which changes the colors and outlines. See the API documentation for more details.

The `scale` option scales the output of the `mpl` and `latex` renderers.

```
circuit = QuantumCircuit(1)
for _ in range(10):
    circuit.h(0)
# limit line length to 40 characters
circuit.draw(output="text", fold=40)
```

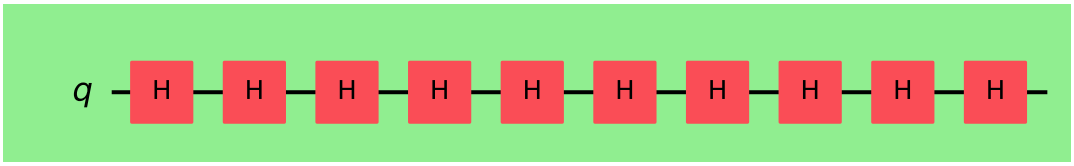
Output:

```
      ┌───┐┌───┐┌───┐┌───┐┌───┐┌───┐┌───┐»
q: ┤ H ├┤ H ├┤ H ├┤ H ├┤ H ├┤ H ├┤ H ├»
      └───┘└───┘└───┘└───┘└───┘└───┘└───┘»
«
«q: ┤ H ├┤ H ├┤ H ├
«     └───┘└───┘└───┘
«
```

```python
# Change the background color in mpl

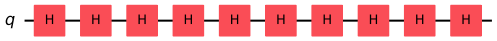
style = {"backgroundcolor": "lightgreen"}
circuit.draw(output="mpl", style=style)
```

Output:




```python
# Scale the mpl output to 1/2 the normal size
circuit.draw(output="mpl", scale=0.5)
```

Output:



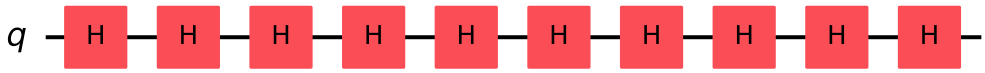
Standalone circuit-drawing function

If you have an application where you prefer to draw a circuit with a self-contained function instead of as a method of a circuit object, you can directly use the `circuit_drawer()` function, which is part of the public stable interface from `qiskit.visualization`. The function behaves identically to the `circuit.draw()` method, except that it takes in a circuit object as a required argument.

```python
from qiskit.visualization import circuit_drawer

circuit_drawer(circuit, output="mpl", plot_barriers=False
```

Output:



Next steps

Recommendations

- See an example of circuit visualization in the Grover's Algorithm tutorial.
- Visualize simple circuits using IBM Quantum Composer.
- Visualize circuit timing.
- Review the Qiskit visualizations API documentation.

Was this page helpful?

Yes No

Report a bug, typo, or request content on GitHub ↗.

© IBM Corp., 2017-2026